PetGuard: Animal Emergency & Care Services

Mobile Application Development Project

📱 PROJECT OVERVIEW

Project Title

PetGuard - Animal Emergency Response and Care Services Mobile Application

Project Type

React Native Mobile Application Development (Cross-Platform: iOS & Android)

Project Duration

4 Weeks (Sprint-based development)

Team Size

5 Students (Individual module ownership with collaborative integration)

🎯 PROJECT OBJECTIVES

Primary Goals

1. Develop a functional mobile application for animal emergency reporting and service requests
2. Implement intelligent scenario identification for different animal care services
3. Create an emergency alert system with priority routing
4. Build a user-friendly interface for critical information collection
5. Integrate GPS location services and multimedia capabilities
6. Establish secure user authentication and session management

Learning Outcomes

- Master React Native mobile development

- Understand component-based architecture
- Implement state management in mobile applications
- Work with device APIs (GPS, Camera, Notifications)
- Practice collaborative software development
- Apply UI/UX best practices for mobile interfaces
- Integrate frontend with backend services
- Implement real-time emergency response workflows

📖 PROJECT DESCRIPTION

Problem Statement

Animals in distress require immediate assistance, but coordinating emergency response and care services is challenging. Citizens need a streamlined way to report animal emergencies, request services, and connect with animal care centers. Current systems lack:

- Quick emergency reporting mechanisms
- Intelligent scenario classification
- Location-based service routing
- Multimedia documentation capabilities
- Centralized service request management

Solution

PetGuard is a comprehensive mobile application that provides:

1. **Emergency Response System**
    - Immediate reporting for sick animals, car accidents, and animal cruelty cases
    - Automated priority classification
    - Real-time GPS location tracking
    - Photo documentation capabilities
    - Emergency team dispatch coordination
2. **Non-Emergency Services**

    - Vaccination appointment scheduling
    - Animal adoption and surrender services
    - Sterilization/spay-neuter program access
    - General animal welfare consultations

3. **Intelligent Scenario Identification**
    - Six distinct service categories
    - Automatic emergency vs. non-emergency classification
    - Context-aware information collection
    - Priority-based routing and response

4. **Critical Information Gathering**
    - GPS-based location detection
    - Photo/video upload for visual assessment
    - Contact information collection
    - Transport capability assessment
    - Detailed situation description

🏗️ APPLICATION ARCHITECTURE

Technology Stack

- **Frontend Framework:** React Native
- **UI Components:** React Native + Lucide React Icons
- **Styling:** Tailwind CSS (via NativeWind) or StyleSheet
- **State Management:** React Hooks (useState, useEffect, useContext)
- **Navigation:** React Navigation (optional for advanced routing)
- **Device APIs:**
    - Geolocation API
    - Camera/Image Picker API
    - Push Notifications (future)

- **Backend Integration:** REST API (to be developed)
- **Storage:** AsyncStorage for local data persistence

Application Flow

```
Splash Screen (3s auto-redirect)
  ↓
Login/Register Screen
  ↓
Home Dashboard (Service Selection)
  ↓
Emergency Detection (if applicable)
  ↓
Information Collection Form
  ↓
Request Submission
  ↓
Confirmation Screen
```

Key Features

1. User Authentication

- Email/password registration
- Secure login with session management
- Guest access option
- Password recovery functionality
- Profile management

2. Service Categories

Emergency Services (Red/Orange indicators):

- Sick Animal - Immediate veterinary attention needed
- Car Accident - Animal hit by vehicle
- Animal Cruelty - Abuse/neglect reporting

Non-Emergency Services (Blue/Green/Purple indicators):

- Vaccination - Schedule immunization appointments
- Adoption - Adopt or surrender animals
- Sterilization - Spay/neuter services

3. Emergency Alert System

- Visual warning screens with pulsing animations
- Priority notification to response teams
- Estimated arrival time display
- Critical information checklist
- Auto-escalation for life-threatening situations

4. Data Collection

- **Location:** GPS coordinates or manual address entry
- **Visual Documentation:** Photo/video capture or upload
- **Contact Details:** Name and phone number
- **Transport Status:** Can user bring animal vs. needs pickup
- **Additional Context:** Free-text description

5. Confirmation & Tracking

- Request summary display
- Unique request ID generation
- Estimated response time
- Contact confirmation
- Return to dashboard option

👥 STUDENT TEAM ASSIGNMENTS

STUDENT 1: Authentication Module 🔐

Responsibilities

Develop complete user authentication system including splash screen, login, and registration functionality.

Components to Build

1. SplashScreen.js

- Animated app logo and branding
- 3-second auto-redirect timer
- Loading animation
- Gradient background design

2. LoginScreen.js

- Email and password input fields
- Input validation (email format, password strength)
- "Remember Me" checkbox functionality
- "Forgot Password" link
- Guest access button
- Error handling and user feedback
- Navigation to registration screen

3. RegisterScreen.js

- Full name, email, phone number, password fields
- Password confirmation matching
- Terms & Conditions acceptance checkbox
- Form validation (required fields, email format, phone format)
- Password strength indicator
- Success/error message display

- Navigation to login screen

4. AuthService.js

- User registration API calls
- User login API calls
- Session token management
- Local storage of user credentials
- Logout functionality

Deliverables

1. Three functional screen components (Splash, Login, Register)
2. Authentication service with API integration
3. Form validation utility functions
4. User session management logic
5. Unit tests for authentication flows
6. Documentation of authentication API endpoints
7. Screen navigation implementation

Technical Requirements

- Input validation with real-time feedback
- Secure password handling (no plain text storage)
- Responsive design for different screen sizes
- Accessibility features (labels, focus management)
- Error handling with user-friendly messages

STUDENT 2: Home & Navigation Module 🏠

Responsibilities

Create the main dashboard, service selection interface, and overall app navigation system.

Components to Build

1. HomeScreen.js

- User welcome header with profile info
- Logout button with confirmation
- Emergency alert banner (prominent placement)
- Six service scenario cards with:
  - Custom icons for each service
  - Color-coded categories
  - Emergency badges for urgent services
  - Descriptive text
  - Tap/press interaction
- Grid/list layout optimization

2. NavigationController.js

- Screen routing logic
- Navigation state management
- Back button handling
- Deep linking support (optional)
- Screen transition animations
- Navigation history tracking

3. UserProfileScreen.js

- Display user information
- Edit profile functionality
- View request history
- Settings/preferences
- Account deletion option

4. scenarioConfig.js

- Centralized scenario definitions
- Icons, colors, descriptions
- Emergency classification
- Service category metadata

Deliverables

1. Fully functional home dashboard
2. Navigation system connecting all screens
3. User profile screen with edit capabilities
4. Scenario configuration file
5. Responsive card layout component
6. Navigation documentation
7. User flow diagrams

Technical Requirements

- Touch-optimized UI elements
- Smooth screen transitions
- Consistent navigation patterns
- State persistence across navigation
- Performance optimization for card rendering

STUDENT 3: Emergency Module 🚨

Responsibilities

Build the emergency detection, classification, and alert system for critical animal situations.

Components to Build

1. EmergencyWarningScreen.js

- Full-screen emergency alert interface

- Pulsing/animated warning indicators
- Critical information checklist display
- Large, prominent action buttons
- Emergency scenario description
- Cancel option with confirmation
- Auto-alert animation

2. EmergencyAlertSystem.js

- Emergency scenario detection logic
- Automatic classification (sick, accident, cruelty)
- Visual alert triggering
- Audio alert integration (optional)
- Priority level assignment
- Admin/dispatch notification system
- Emergency timer/countdown

3. emergencyScenarios.js

- Complete scenario database
- Emergency vs. non-emergency flags
- Severity levels (critical, high, medium, low)
- Required information fields per scenario
- Response time estimates
- Dispatch protocols

4. AlertNotificationComponent.js

- In-app alert banners
- Push notification setup (future integration)
- Alert sound management
- Vibration patterns for emergencies

Deliverables

1. Emergency warning screen with animations
2. Emergency classification algorithm
3. Alert notification system
4. Scenario database with complete metadata
5. Emergency response protocol documentation
6. Testing suite for emergency flows
7. Alert customization options

Technical Requirements

- <100ms emergency detection response time
- Clear visual hierarchy for critical information
- Accessibility compliance (high contrast, screen readers)
- Offline emergency mode support
- Battery-efficient alert system

STUDENT 4: Information Collection Module 📝

Responsibilities

Develop comprehensive data collection forms with GPS, camera, and input validation capabilities.

Components to Build

1. InfoFormScreen.js

- Dynamic form layout based on scenario type
- GPS location input with manual override
- Contact information fields
- Transport capability selector (Yes/No buttons)
- Additional information textarea

- Photo upload section
- Form validation and error display
- Required field indicators
- Submit button with loading state

2. LocationService.js

- GPS coordinate detection
- Location permission requests
- Address geocoding (coordinates to address)
- Manual address input handling
- Location accuracy indicator
- Error handling for GPS failures
- Location caching

3. PhotoUploadComponent.js

- Camera access and photo capture
- Gallery/photo library access
- Image preview functionality
- Multiple photo upload support
- Image compression before upload
- Photo deletion option
- Upload progress indicator

4. FormValidation.js

- Real-time field validation
- Phone number formatting
- Email validation
- Required field checking
- Custom validation rules per scenario

- Error message generation

Deliverables

1. Complete information collection form
2. GPS location service with fallback options
3. Photo capture and upload component
4. Form validation utility library
5. Input sanitization functions
6. Accessibility-compliant form elements
7. Offline data collection capability (save locally)

Technical Requirements

- Device permission handling (location, camera)
- Graceful degradation if GPS unavailable
- Image optimization (max 2MB per photo)
- Form state persistence (draft saving)
- Cross-platform compatibility (iOS/Android)

STUDENT 5: Confirmation & Backend Integration ✅

Responsibilities

Build request confirmation interface and handle all backend API communications.

Components to Build

1. ConfirmationScreen.js

- Success animation and checkmark
- Request summary display
- Emergency vs. non-emergency confirmation messaging
- Request ID/reference number

- Estimated response time
- Contact confirmation details
- Next steps guidance
- Return to home button
- Share request option

2. APIService.js

- RESTful API client setup
- User registration endpoint
- User login endpoint
- Service request submission
- Photo upload to server
- Request history retrieval
- Request status updates
- Error handling and retry logic

3. DataModels.js

- ServiceRequest model structure
- User model structure
- Response/Notification model
- Data validation schemas
- Type definitions
- Model serialization/deserialization

4. LocalStorageService.js

- AsyncStorage implementation
- Save user session
- Cache request history
- Store draft requests

- Clear cache functionality

5. API Documentation

- Complete endpoint documentation
- Request/response examples
- Authentication requirements
- Error code definitions

Deliverables

1. Confirmation screen with success animations
2. Complete API service layer
3. Data models with TypeScript types (optional)
4. Local storage service
5. API integration documentation
6. Error handling system
7. Request/response logging
8. Backend API endpoint specifications

Technical Requirements

- JWT token authentication
- Automatic token refresh
- Request timeout handling (30s max)
- Offline request queueing
- Data encryption for sensitive info
- API versioning support
- Rate limiting compliance

📋 SHARED DELIVERABLES (All Students)

1. Code Quality

- Clean, well-commented code
- Consistent naming conventions
- Modular component structure
- No code duplication
- ESLint/Prettier compliance

2. Documentation

- Component API documentation
- Function parameter descriptions
- Usage examples
- Integration guides
- Troubleshooting section

3. Testing

- Unit tests for core functions (minimum 80% coverage)
- Integration tests for API calls
- UI component tests
- Error scenario testing
- Performance testing

4. Version Control

- Regular Git commits with meaningful messages
- Feature branch workflow
- Pull request reviews
- Merge conflict resolution
- Git documentation

📦 SHARED RESOURCES

Common Files (All Students Use)

1. styles.js / theme.js

```javascript
// Colors
const Colors = {
  primary: '#3B82F6',     // Blue
  danger: '#EF4444',      // Red
  success: '#10B981',     // Green
  warning: '#F59E0B',     // Orange
  purple: '#8B5CF6',
  // ... all app colors
};

// Typography
const Typography = {
  h1: { fontSize: 32, fontWeight: 'bold' },
  h2: { fontSize: 24, fontWeight: 'bold' },
  // ... font styles
};

// Spacing
const Spacing = {
  xs: 4, sm: 8, md: 16, lg: 24, xl: 32
};
```

2. constants.js

```javascript
export const API_BASE_URL = 'https://api.petguard.com';
export const APP_VERSION = '1.0.0';
export const MAX_PHOTO_SIZE = 2 * 1024 * 1024; // 2MB
export const GPS_TIMEOUT = 10000; // 10 seconds
// ... all app constants
```

3. utils.js

```javascript
// Validation helpers
export const validateEmail = (email) => { /* ... */ };
export const validatePhone = (phone) => { /* ... */ };
export const formatDate = (date) => { /* ... */ };
// ... utility functions
```

📅 PROJECT TIMELINE

Week 1: Individual Module Development

- Day 1-2: Environment setup, project planning
- Day 3-5: Core component development
- Day 6-7: Initial testing and debugging

Week 2: Integration & Testing

- Day 8-9: Module integration
- Day 10-11: End-to-end testing
- Day 12-13: Bug fixes and refinement
- Day 14: Integration review meeting

Week 3: Enhancement & Polish

- Day 15-17: UI/UX improvements
- Day 18-19: Performance optimization
- Day 20-21: Final testing and documentation

Week 4: Presentation & Deployment

- Day 22-23: Presentation preparation
- Day 24: Team presentation and demo

- Day 25-26: Code review and final submission
- Day 27-28: Project retrospective

🎯 INTEGRATION REQUIREMENTS

Data Flow Between Modules

Student 1 → Student 2: User authentication state

- `user` object (name, email, token)
- `isAuthenticated` boolean
- `logout()` function

Student 2 → Student 3: Scenario selection

- `selectedScenario` object
- `isEmergency` boolean flag
- Navigation trigger to emergency screen

Student 3 → Student 4: Emergency classification

- `emergencyAlert` boolean
- `priority` level
- Required information fields

Student 4 → Student 5: Form data

- `formData` object with all collected info
- `photos` array
- Validation status

Student 5 → All: API responses

- Success/error status
- Request confirmation

- Server-side validation errors

Integration Meetings

- **Week 1 End:** Initial integration check
- **Week 2 Middle:** Full integration testing
- **Week 3 Start:** Final integration review

📊 GRADING CRITERIA (100 Points Total)

Individual Component Development (40 points)

- Functionality: 20 points
- Code Quality: 10 points
- Documentation: 5 points
- Testing: 5 points

Integration (25 points)

- Successful module integration: 15 points
- Data flow correctness: 10 points

Team Collaboration (15 points)

- Git usage and commits: 5 points
- Code reviews and feedback: 5 points
- Communication and meetings: 5 points

Final Presentation (20 points)

- Live demo: 10 points
- Technical explanation: 5 points
- Q&A handling: 5 points

🚀 DEPLOYMENT & SUBMISSION

Final Submission Package

1. Complete source code (GitHub repository)
2. Installation and setup guide
3. User manual / how-to guide
4. API documentation
5. Testing report
6. Team contribution log
7. Presentation slides
8. Demo video (5-10 minutes)

Deployment Options

- **Android:** APK file for testing
- **iOS:** TestFlight build (if available)
- **Web:** Expo web build (fallback)

📚 RECOMMENDED RESOURCES

React Native Documentation

- Official React Native Docs: https://reactnative.dev
- React Navigation: https://reactnavigation.org
- AsyncStorage: https://react-native-async-storage.github.io

UI/UX Resources

- Material Design Guidelines
- iOS Human Interface Guidelines
- Lucide React Icons: https://lucide.dev

APIs & Services

- Geolocation API documentation
- React Native Camera documentation
- REST API best practices

🤝 SUPPORT & COMMUNICATION

Team Communication Channels

- Weekly team meetings (mandatory)
- Slack/Discord for daily updates
- GitHub for code collaboration
- Shared Google Drive for documentation

Instructor Office Hours

- Available for technical questions
- Code review sessions
- Integration support
- Deployment assistance

⚠️ IMPORTANT NOTES

Academic Integrity

- All code must be original or properly attributed
- External libraries must be approved
- No code sharing between teams
- Plagiarism will result in project failure

Device Testing Requirements

- Test on at least 2 different devices
- Test both iOS and Android (if possible)
- Test different screen sizes
- Report platform-specific issues

Backup & Version Control

- Daily Git commits required
- Regular backups of work
- Tag major milestones
- Document all major changes

📞 EMERGENCY CONTACT

Project Instructor: Professor Elizabeth
Email: elizabeth.perez@uta.edu
Office: ERB 551
Office Hours: MW 2-4 PM or by appointment

✅ PROJECT SUCCESS CRITERIA

The project will be considered successful when:

1. All authentication flows work correctly
2. All six service scenarios are functional
3. Emergency alerts trigger properly
4. GPS location and photo upload work
5. Form validation prevents invalid submissions
6. API integration handles all CRUD operations
7. App runs without crashes on test devices
8. UI is intuitive and user-friendly

9. All required documentation is complete
10. Team successfully demonstrates the app

Project Start Date: [To be announced]
Final Submission Deadline: [To be announced]
Presentation Date: [To be announced]

Good luck to all teams! 🐾